

DeliverEase - Restaurant Delivery Management System

README Documentation

Table of Contents

1. Project Idea
2. Key Features
3. Technology Stack
4. Design Guidelines
5. Database Schema
6. How AJAX & APIs Work
7. User Roles & Functionality
8. Project Structure
9. Installation & Setup
10. API Endpoints
11. Future Enhancements

1. Project Idea

Many small and medium-sized restaurants still rely on manual order-taking (phone calls or paper notes), leading to errors, delays, and poor customer experience. DeliverEase solves this by offering a centralized digital platform where:

- Customers can browse the menu, manage a cart, and place orders online.
- Kitchen staff (chefs) see incoming orders in real time and update their status as they progress.
- Restaurant managers monitor overall performance through live statistics (revenue, order counts, delayed orders) and can intervene in any order.

The result is a faster, more accurate, and fully transparent order management system.

2. Key Features

Customer Features

- User-friendly menu with categories and images
- Client-side cart (stored in localStorage) – add, remove, update quantities instantly
- Checkout with two payment methods: Cash on delivery / Online payment (with receipt upload)
- Order placement via AJAX – no page refresh, instant confirmation
- Personal order tracking – view all past orders and their current status

Chef Features

- Kitchen dashboard showing all active orders (pending / preparing / ready)
- Color-coded cards: yellow = pending, blue = preparing, green = ready
- One-click status updates (Start Preparing -> Mark as Ready) using AJAX
- Page automatically refreshes the order list after each action

Manager Features

- Real-time statistics dashboard: Today's orders, Monthly orders, Total revenue, Number of delayed/cancelled orders
- Full order management – filter orders by status, change any order's status
- No customer names displayed – privacy-focused design
- Auto-refresh stats every 30 seconds via AJAX

3. Technology Stack

Layer	Technology
Backend	Django 4.x (Python) – REST-like APIs returning JSON
Frontend	HTML5, CSS3, Bootstrap 5, Vanilla JavaScript (ES6)
Database	SQLite3 (development), easily switchable to PostgreSQL
AJAX	XMLHttpRequest – used for all asynchronous operations
Storage	localStorage for cart persistence
Icons	Bootstrap Icons

4. Design Guidelines

Color Palette :

Primary #E1AD01	Secondary #2D3142	Accent #4CAF50	Background #F8F9FA	Text #212529
---------------------------	-----------------------------	--------------------------	------------------------------	------------------------

Typography:

- Headings: Poppins
- Body: Open Sans

5. Database Schema

User (custom user with role)

└─ role: customer | chef | manager

Category

└─ name, slug

MenuItem

└─ category (FK), name, description, price, image, available

Order

└─ customer (FK), delivery_address, total_price, status, created_at

OrderItem

└─ order (FK), menu_item (FK), quantity, item_price

```
|
Payment
├─ order (OneToOne), method (cod/online), is_paid, receipt,
paid_at
```

6. How AJAX & APIs Work

Instead of traditional form submissions that reload the entire page, DeliverEase leverages AJAX to communicate with the backend asynchronously.

Flow Example (Placing an Order):

1. Customer builds a cart (stored in localStorage)
2. On checkout, JavaScript (checkout.js) gathers cart data, address, and payment method
3. An XMLHttpRequest POST is sent to `/api/place-order/` with a JSON payload
4. The Django view validates, creates Order and OrderItem records, returns JSON response (`{ success: true, order_id: 123 }`)
5. Frontend displays success message and clears cart – no full page reload

7. User Roles & Functionality

Role	Accessible Pages	Main Actions
Customer	<code>/menu/</code> , <code>/cart/</code> , <code>/checkout/</code> , <code>/track/</code>	Browse menu, manage cart, place order, track order history
Chef	<code>/chef/</code>	View active orders, change status (preparing -> ready)
Manager	<code>/manager/</code>	View dashboard statistics, filter/change any order status, see recent orders

8. Project Structure

```
Project/
├─ DeliverEase/                # Main Django app (Changed from
core/ to DeliverEase/)
```

```

|   ├── models.py                # User, Category, MenuItem, Order,
OrderItem, Payment
|   ├── views.py                 # All views (pages + APIs)
|   ├── urls.py                  # URL configuration
|   └── admin.py                 # Custom admin setup
└── templates/                   # HTML files
    ├── login.html, register.html, menu.html, cart.html,
checkout.html
    ├── order_tracking.html, chef_dashboard.html,
manager_dashboard.html, base.html
    └── static/
        ├── js/                  # AJAX-driven JavaScript files
        ├── login.js, register.js, menu.js, cart.js, checkout.js,
chef.js, manager.js
        └── media/                # Uploaded images (menu items,
receipts)
    ├── manage.py
    └── README.md

```

9. Installation & Setup

1. Clone the repository

```
git clone https://github.com/your-team/deliverease.git
cd deliverase
```

2. Create & activate a virtual environment

```
python -m venv venv
source venv/bin/activate
```

3. Install dependencies

```
pip install django
```

4. Apply migrations

```
python manage.py makemigrations
python manage.py migrate
```

5. Create a superuser (admin)

```
python manage.py createsuperuser
```

6. Run the development server

```
python manage.py runserver
```

10. API Endpoints

Method	URL	Description	Auth
POST	/api/login/	User login (returns JSON)	No
POST	/api/register/	Customer registration	No
POST	/api/place-order/	Place a new order	Yes
POST	/api/order/<id>/status/	Update order status	Yes

11. Future Enhancements

- Online payment gateway integration (Stripe / PayPal)
- Real-time order tracking using WebSockets
- Push notifications for chefs and customers
- Mobile app using React Native or Flutter
- Advanced analytics and charts (e.g., Chart.js)
- Email notifications for order status changes